Series temporales

Series temporales

Principales tipos de Ensemble Models:

Definicion: Una serie temporal es una sucesión de observaciones de una variable tomadas en varios instantes de tiempo.

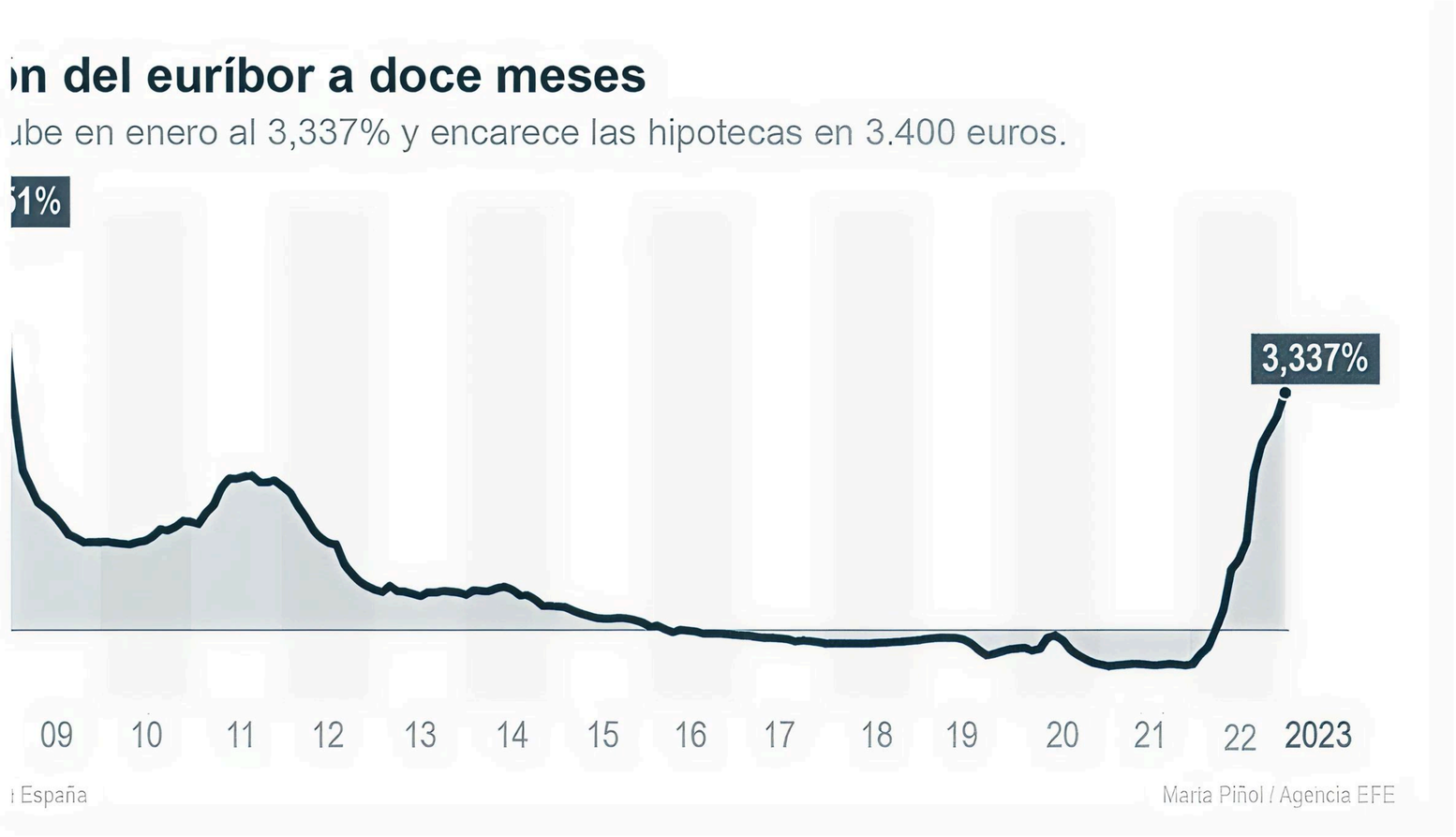
Análisis: El objetivo es estudiar los cambios en esa variable con respecto al tiempo y tratar de predecir sus valores futuros.
Importante: Algunos modelos de esta familia de modelos no está incluida en Scikit Learn (xGBoost, Catboost y LGBM por ejemplo)

Ejemplos de series temporales:

- Economía: PIB, Inflación, tasa de desempleo, número de matriculaciones,
- Demografía: nacimientos anuales, tasa de dependencia, etc.
- Meteorología: temperaturas máximas, medias o mínimas, precipitaciones

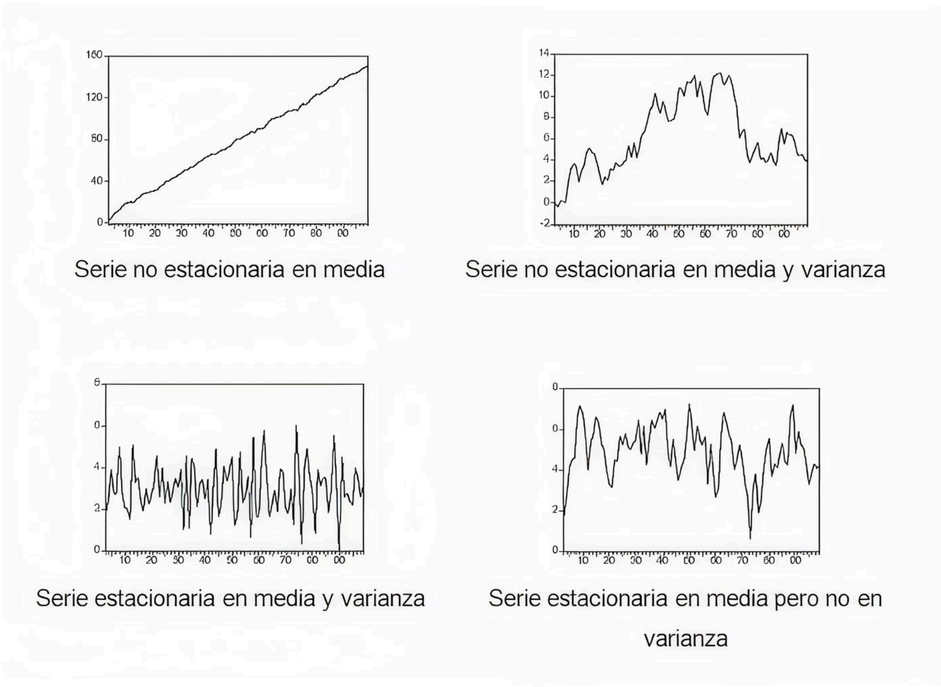
Series temporales: ejemplo

Series temporales: ejemplo



Series temporales: características

Estacionariedad: Una serie es estacionaria si la media y la varianza se mantienen constantes a lo largo del tiempo.



La estacionariedad es una suposición importante en muchos modelos de series temporales, ya que permite el uso de modelos más simples que son más fáciles de interpretar, analizar y de predecir.

Series temporales: caracteristicas

Autocorrelación

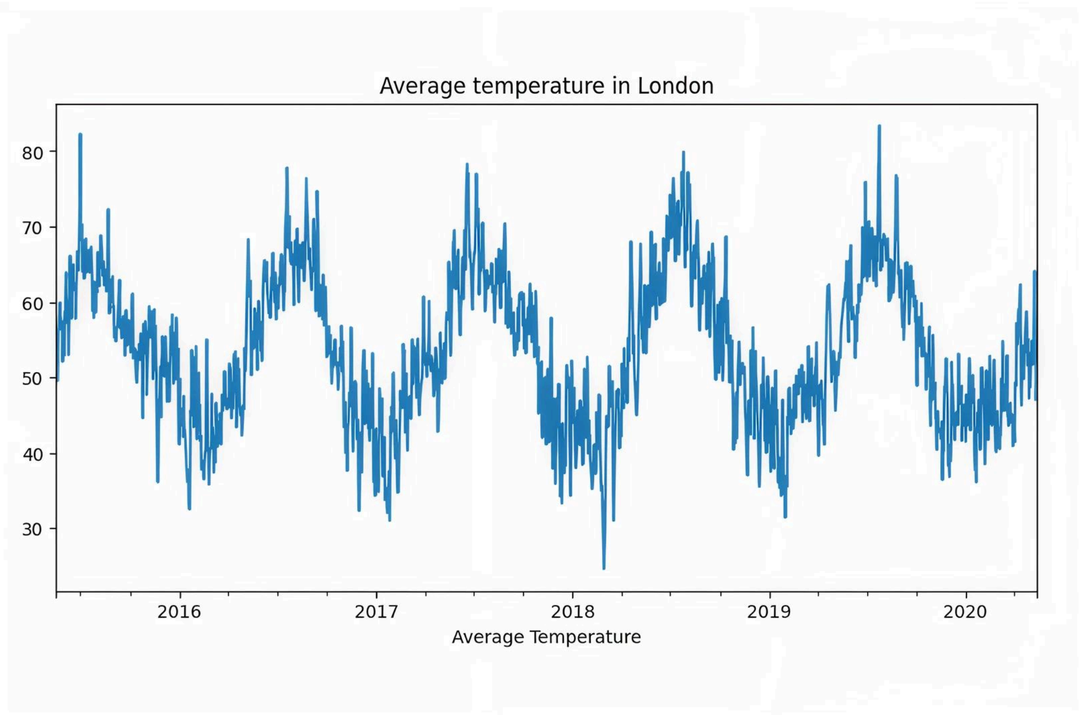
La autocorrelación se refiere al grado de correlación entre una serie temporal y una versión rezagada de sí misma.

- Autocorrelación alta indica que el valor actual de la serie temporal está relacionado con valores pasados.
- Autocorrelación baja indica que el valor actual no está relacionado con valores pasados.

Estacionalidad: La estacionalidad se refiere a patrones repetitivos en una serie de tiempo que ocurren a intervalos regulares, como mensual, trimestral o anualmente.

Descomposición: La descomposición es una técnica para separar una serie de tiempo en sus componentes individuales, como la tendencia, la estacionalidad y el ruido.

$$X_t = T_t + E_t + R_t$$



Series temporales: casos de uso

- Predecir el número altas de líneas ADSL con el objetivo de dimensionar las centralitas de telefónica.
- Predecir el número de periódicos a dejar en el punto de venta
- Estimar la demanda de hipotecas
- Predecir la evolución de los saldos en cuenta
- Forecast financiero

Preguntas ante un proyecto de forecasting

1. ¿Cuántos datos tengo disponibles? Cuántos más suele ser mejor.
   En series temporales mensuales por ejemplo se suele pensar en un mínimo de 12 observaciones

2. ¿Cuál es el horizonte de mis previsiones? Corto medio, largo plazo?
   A corto plazo suele ser más fácil y confiable

3. ¿Nuestras previsiones deben ser estáticas o actualizarse en el tiempo?

Series temporales: modelos

1

ARIMA

(Autoregressive Integrated Moving Average) Es uno de los modelos más populares en el análisis de datos temporales porque puede capturar relaciones pasadas para proyectar tendencias futuras y manejar datos no estacionarios al incluir diferenciación.

2

SARIMAX

(Seasonal AutoRegressive Integrated Moving Average with eXogenous regressors) Es una generalización de ARIMA que se emplea cuando hay estacionalidad o variables exógenas a la propia serie. Gracias a su flexibilidad, permite incorporar efectos adicionales que no pueden ser capturados por modelos ARIMA tradicionales.

3

Alisado exponencial

El alisado exponencial es un método para suavizar una serie temporal al asignar pesos decrecientes a los datos pasados. Esto significa que los datos más recientes tienen mayor influencia en las predicciones que los más antiguos. Es ideal para detectar patrones como tendencias o estacionalidades con un enfoque menos complejo que ARIMA.

4

Prophet

Prophet, desarrollado por Facebook se basa en la idea de descomponer los datos en componentes fundamentales para facilitar las predicciones. Está diseñado para manejar series con patrones de tendencia, estacionalidad y ruido, ofreciendo una solución robusta incluso para datos con intervalos irregulares, valores faltantes o cambios estacionales complejos.

Series temporales: ARIMA

Requisitos principales

1. **Estacionariedad**
   - La serie temporal debe ser estacionaria, lo que significa que su media y varianza no cambian con el tiempo.
   - Si no es estacionaria, se puede transformar mediante el componente de diferenciación (el parámetro d).

2. **Autocorrelaciones**
   - ARIMA asume que existe una relación entre los valores actuales y pasados de la serie, así como los errores.

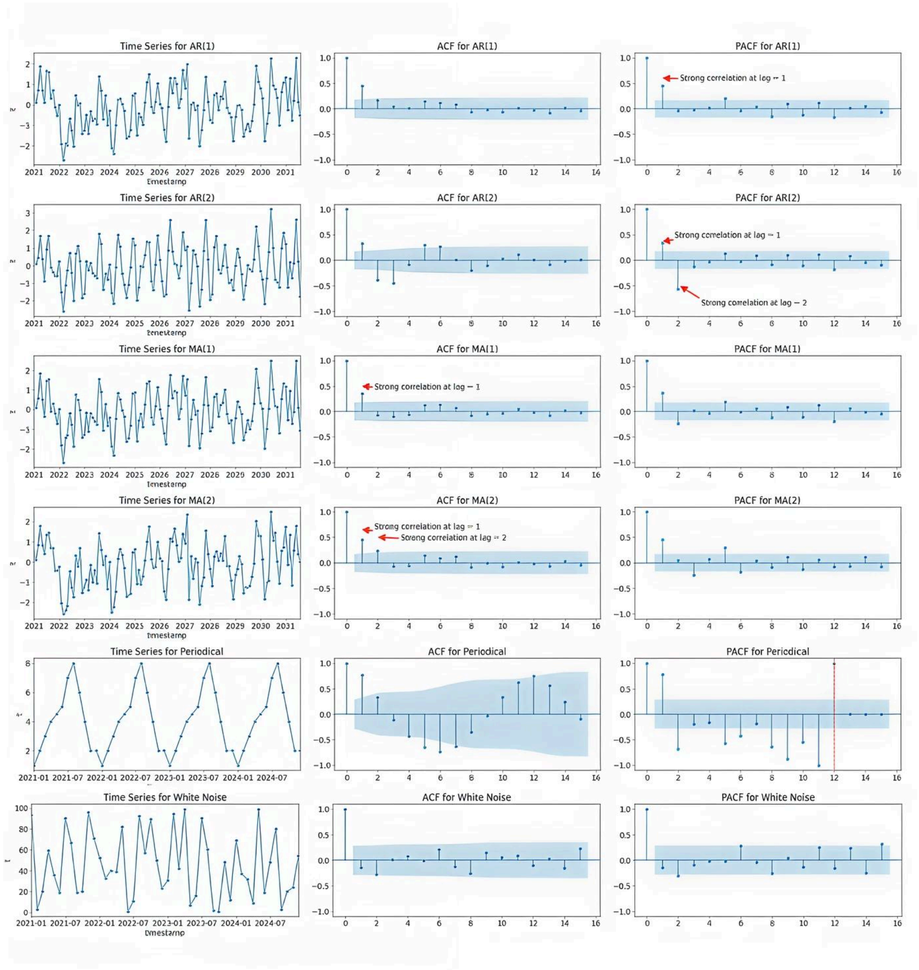
Componentes

1. **AR (Autoregressive)**
   - Representa la parte autorregresiva del modelo.
   - Indica que los valores actuales de la serie temporal dependen linealmente de los valores pasados (lags).
   - Se define por un parámetro p, que especifica el número de retardos incluidos en el modelo.

2. **I (Integrated):**
   - Representa la parte integrada, que se utiliza para convertir una serie no estacionaria en estacionaria mediante diferenciación.
   - El parámetro d indica el número de diferenciaciones necesarias para hacer que la serie sea estacionaria.

3. **MA (Moving Average)**
   - Representa la parte de promedios móviles del modelo.
   - Modela la relación entre un valor y el error (residual) de los valores pasados.
   - Se define por el parámetro q, que determina el número de términos de error incluidos.

Series temporales: Diagnóstico con ACF y PACF



Series temporales: pmdarima

El paquete pmdarima estima diferentes modelos ARIMA y se queda con el que funciona mejor de acuerdo a un parámetro, por ejemplo: Akaike Information Criterion:

AIC: Evalúa la calidad del modelo penalizando la complejidad. Se usa comúnmente para seleccionar modelos ARIMA.

`start_p, start_q:` Valores iniciales para los términos AR (p) y MA (q).
`max_p, max_q:` Valores máximos para p y q que el algoritmo probará.
`d:` Orden de diferenciación. Si es None, se determina automáticamente con pruebas estadísticas.
`seasonal:` Si True, busca un modelo SARIMA con componentes estacionales.
`m:` Periodo de estacionalidad (por ejemplo, m=12 para datos mensuales).
`stepwise:` Si True, usa un algoritmo más eficiente para encontrar el mejor modelo en lugar de probar todas las combinaciones posibles.
`test:` Método para determinar d (por ejemplo, 'kpss', 'adf').
`information_criterion:` Criterio para seleccionar el mejor modelo ('aic', 'bic', etc.).
`trace:` Si True, muestra el proceso de selección del modelo en la consola.

https://alkaline-ml.com/pmdarima/modules/generated/pmdarima.arima.auto_arima.html

```
model = pm.auto_arima()
print(model.summary)
```

Series temporales: Prophet

Prophet es una biblioteca de Python de código abierto desarrollada por Facebook.

Está basado en un modelo aditivo, donde las tendencias no lineales se ajustan con estacionalidad anual, semanal y diaria, además de efectos de días festivos.

Funciona mejor con series temporales que tienen efectos estacionales fuertes y varias temporadas de datos históricos.

Prophet es resistente a la falta de datos y a cambios en la tendencia, y generalmente maneja bien los valores atípicos.

Prophet es una biblioteca fácil de usar, casi no tiene configuraciones adicionales, y es capaz de obtener predicciones bastante razonables.

Series temporales: Prophet

```
# Crear y entrenar el modelo
model_prp = Prophet()
model_prp.fit(train_prp)

# Generar fechas futuras para predicción
futuro = model_prp.make_future_dataframe(periods=12, freq='MS')  # Predicción para
30 días adicionales

# Hacer predicciones
predictions_prp = model_prp.predict(futuro)
```

Series temporales: Alisado exponencial

El alisado exponencial enfatiza los valores recientes mediante un factor de suavizamiento.
Es útil para capturar tendencias y estacionalidades sin requerir modelos complejos como Prophet o ARIMA.

Tipos de alisado exponencial

- Simple Exponential Smoothing (SES) → Captura solo la tendencia a corto plazo.
- Double Exponential Smoothing (Holt) → Modela tendencia creciente o decreciente.
- Triple Exponential Smoothing (Holt-Winters) → Maneja tendencia + estacionalidad.

```
from statsmodels.tsa.holtwinters import
ExponentialSmoothing


# Ajustar modelo Holt-Winters
ESmodel = ExponentialSmoothing(train,
trend="add", seasonal="add",
seasonal_periods=12)
ESmodel_fit = ESmodel.fit()

# Generar predicciones futuras
ES_predictions = ESmodel_fit.forecast(12)
```

Otras librerías para trabajar con series temporales

Auto_TS

Auto_TimeSeries te permite construir y seleccionar múltiples modelos de series temporales utilizando técnicas como ARIMA, SARIMAX, VAR, modelos descomponibles (tendencia + estacionalidad + festivos) y modelos de aprendizaje automático ensamblados.

https://github.com/AutoViML/Auto_TS

DART

Darts es una biblioteca de Python para la predicción y detección de anomalías en series temporales de forma fácil y amigable para el usuario. Contiene una amplia variedad de modelos, desde clásicos como ARIMA hasta redes neuronales profundas. Todos los modelos de predicción pueden utilizarse de forma uniforme, mediante las funciones fit() y predict(), similar a cómo se hace en scikit-learn.

https://github.com/unit8co/darts

Greykite

La biblioteca Greykite proporciona un marco que facilita el desarrollo de un buen modelo de predicción, incluyendo análisis exploratorio de datos, preprocesamiento de valores atípicos y anomalías, extracción y creación de características, búsqueda en cuadrícula (grid search), evaluación, comparación de modelos (benchmarking) y visualización gráfica

https://github.com/linkedin/greykite

Merlion

Merlion ofrece un framework completo de aprendizaje automático que incluye la carga y transformación de datos, la construcción y entrenamiento de modelos, el posprocesamiento de los resultados del modelo y la evaluación del rendimiento.

https://github.com/salesforce/Merlion

Sktime

Sktime es una biblioteca para el análisis de series temporales en Python. Ofrece una interfaz unificada para múltiples tareas de aprendizaje con series temporales. Actualmente, incluye clasificación, regresión, agrupamiento (clustering), anotación y predicción (forecasting) de series temporales. Cuenta con algoritmos específicos para series temporales y herramientas compatibles con scikit-learn para construir, ajustar y validar modelos.

https://github.com/sktime/sktime